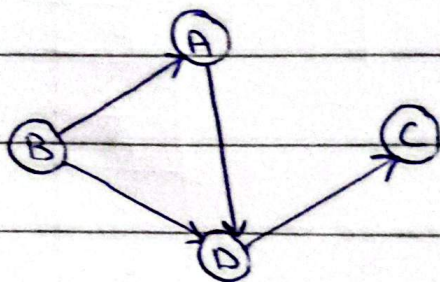


Topological Sort or Topological Ordering:

- It is a linear ordering of graph vertices such that for every directed edge uv from vertex u to vertex v , u comes before v in the ordering.
- Applicable on DAG (Direct Acyclic Graph).
- Linear running time complexity.
- Topological sort is not unique.

Example:



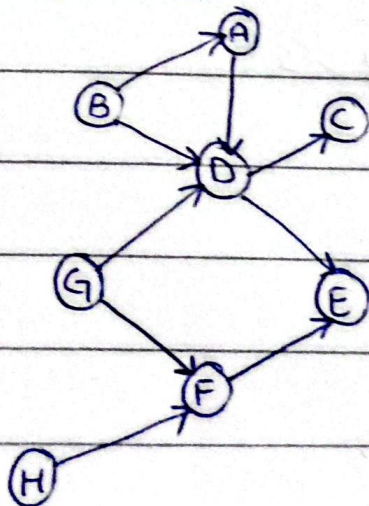
⇒ Directed and Acyclic Graph.

Topological order :

BADC → valid Topological order

ABCD → Invalid Topological order.

Example 02:



⇒ How we actually write it?

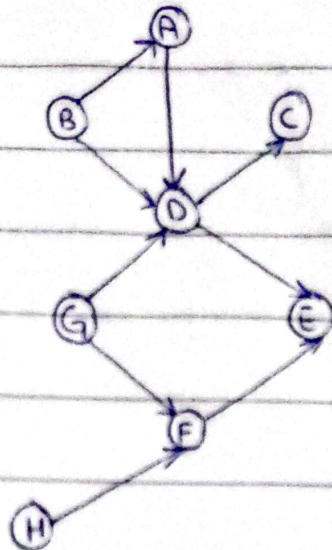
So there are multiple methods:

- DFS
- BFS
- Kahn's Algorithm.

Day: _____

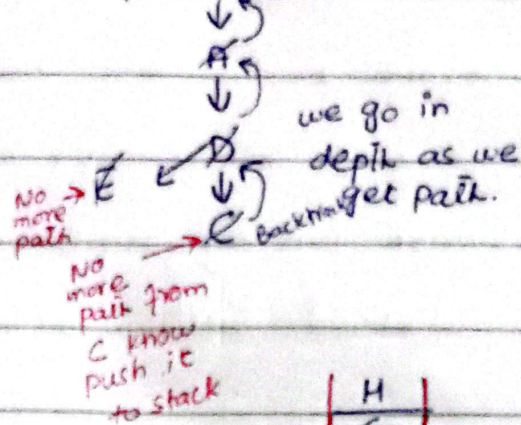
Date: _____

But the most important we discuss first is
DFS (Depth First Search):

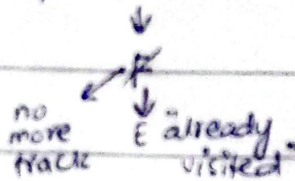


• Let's we start from B

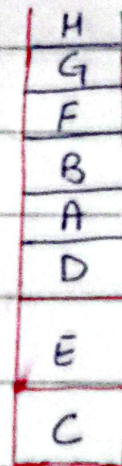
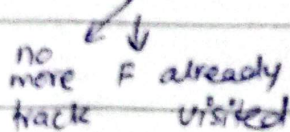
DFS (B):



DFS (G):



DFS (H):



Stack.

last in first come

"Topological order using DFS"

Time complexity:

$$O(V+E)$$

Kahn's Algorithm for topological sort:

↳ uses BFS (Breadth first search) technique.

↳ use Queue data structure.

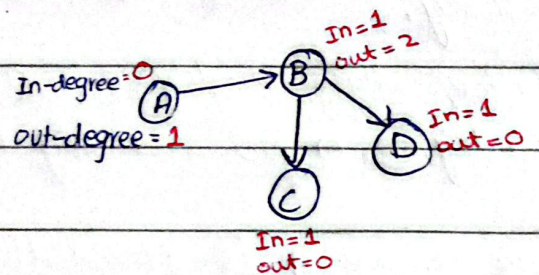
In Kahn's algorithm we have to do The first step is that write the in degree of all the vertex.

what is in degree?

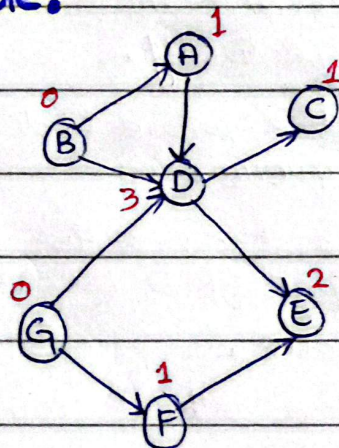
In degree is that how many edges are coming toward vertex.

If there is any vertex then how many edges are coming toward it.

Out degree → how many is going out of that vertex.



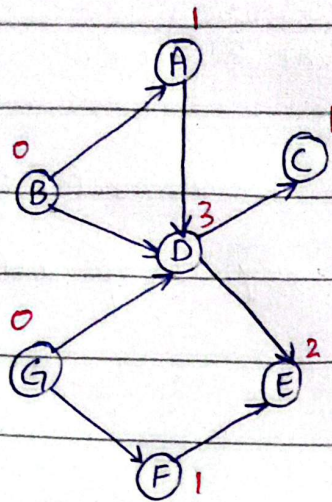
Example:



• First of all write in-degree on each vertex.

"This is the property of acyclic graph that at least one vertex will be such whose in degree is zero" and one of the vertex has out degree zero"

→ Then the graph is acyclic.



First of all add all vertices in queue whose in degree is 0."

B	G	A	F	∅	∅	∅
---	---	---	---	---	---	---

original
Final
Sorting →

B	G	A	F	D	C	E
---	---	---	---	---	---	---

- when we remove B, then its connection with A and D breaks and their in-degrees become 0 and 2 respectively.
- Now any other vertices whose in-degree is 0, yes know A has in-degree 0 so add this to queue.
- Similarly, when we remove G its connection with D and F breaks and their in-degrees become 1 and 0 respectively.
- Now F also becomes a vertex whose in-degree is 0 so add it to queue.
- Now, let's remove A so its connection with D breaks and in-degree of D becomes 0.
- So now add D to queue.
- F removes and its connection with E removes and its in-degree becomes 1.

Day: _____

Date: _____

- No vertex with indegree 0, so know remove next which is D and its connection with C and E breaks and these indegree become 0 and 0 respectively.
- Now C and E have indegree 0 add to the queue.

Topological Sort :

B G A F D C E

Algorithm:

L $\rightarrow$ Empty list that will contain the sorted element.

S $\rightarrow$ Set of all nodes with no incoming edge.

while S is not empty do

 remove a node n from S

 add n to L

 For each node m with an edge e from n to m do

 remove edge e from the graph.

 if m has no other incoming edges then
 insert m into S

if graph has ~~no~~ edges then

 return error (graph has at least one cycle).

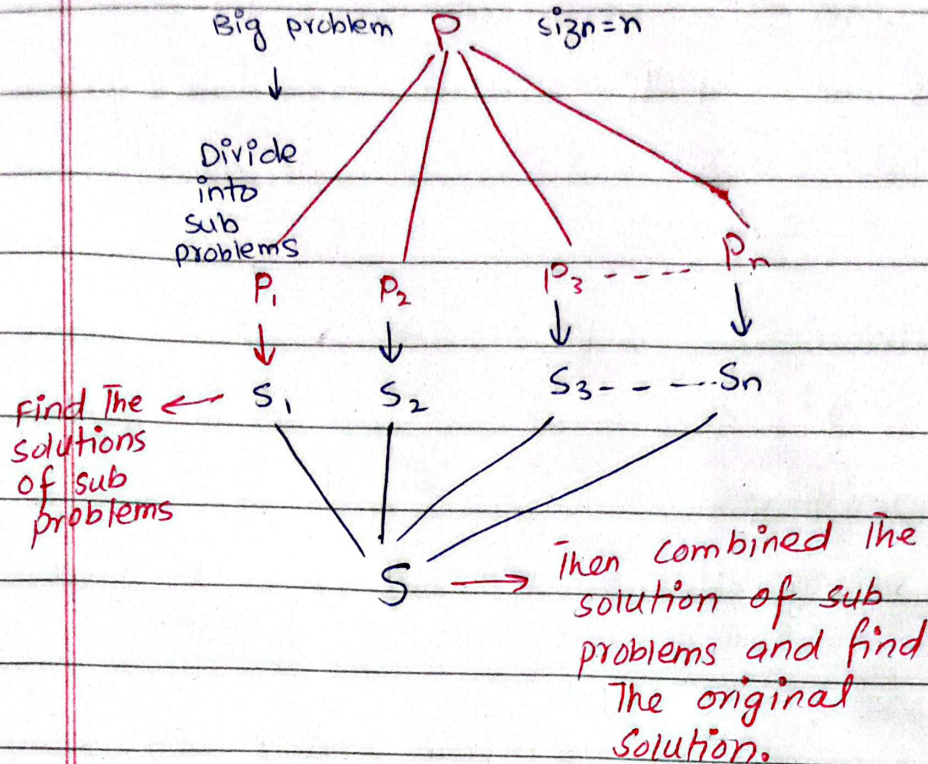
else

 return L (a topologically sorted order).

Time Complexity:

$O(V+E)$.

Divide and Conquer:



- If subproblem is big then again use this strategy for that subproblem (divide and conquer).
- This strategy says, recursively solve a problem if it is too big recursively solve it.
- And if you have broken the problem into subproblem, you must have a method to combine these solutions to make one solution.
If you can't do this you cannot use this strategy.

Day: _____

Date: _____

General Method for DAC strategy:

```
DAC (P) {  
    if (small (P))  
        S(P);  
    else  
        divide P into  $p_1, p_2, p_3, \dots, p_n$   
        Apply DAC ( $p_1$ ), DAC ( $p_2$ ) ----  
        Combine (DAC ( $p_1$ ), DAC ( $p_2$ ) ---- )  
}
```

Problems That use DAC :

- 01- Binary Search
- 02- Finding maximum and minimum
- 03- Merge Sort
- 04- Quick Sort
- 05- Strassen's Matrix Multiplication

Merge Sort:

First of all let see what is Merge?

Merging is the processing of combining two sorted list into a single sorted list.

Step 01:

A: 2 8 15 18
B: 5 9 12 17

- compare i of A with j of B if smaller write it in C and increment i .

C: 2
↑
K

$i++$

- And if not Then write value of B that point j and then increment j .

Step 02:

A: 2 8 15 18
B: 5 9 12 17

Again compare

$8 < 5 \rightarrow \text{No}$

↓
write value of j in C.

C: 2 5
↑
K

$j++$

Step 03:

A: 2 8 15 18
B: 5 9 12 17

Again compare
 i with j
 i is smaller.

C: 2 5 8
↑

Day: _____

Date: _____

$i++$

Step 04:

A: 2 8 15 18

B: 5 9 12 17

Again compare
 i with j
 j is smaller

C: 2 5 8 9 12

$j++$

Step: 05

A: 2 8 15 18

B: 5 9 12 17

Again compare
 i with j
 j is smaller.

C: 2 5 8 9 12 15

$j++$

Step 06:

A: 2 8 15 18

B: 5 9 12 17

C: 2 5 8 9 12 15 17

$i++$

Step 07:

A: 2 8 15 18

B: 5 9 12 17

C: 2 5 8 9 12 15 17

$j++$

Day: _____

Date: _____

step 08:

A: 2 8 15 (18)

B: 5 9 12 17

C: 2 5 8 9 12 15 17 18

- when we complete one list then we ^{write} move all other ~~at~~ remaining element in C.

Final list: (Sorted) → using merging.

C

2

5

8

9

12

15

17

18

(m+n)
elements

⇒ If list A has m elements and list B has n elements Then C has m+n elements.

Time Complexity:

$O(m+n)$.

Algo Merge (A, B, m, n)

$i=1$; $j=1$; $k=1$;

while ($i \leq m$ and $j \leq n$) {

if ($A[i] < B[j]$)

$C[k++] = A[i++]$;

else

$C[k++] = B[j++]$;

for (i ; $i \leq m$; $i++$)

$C[k++] = A[i]$

for (j ; $j \leq n$; $j++$)

$C[k++] = B[j]$

// If there are any remaining elements after comparing, copy the elements and paste it.

How To merge when u have more Than 2 lists?

• 4-Way Merging (N-Way Merging).

A	B	C	D
4	3	8	2
6	5	10	4
12	9	16	18

→ Find smallest element in all of this

Here, it is 2 so add it in final list.

Day: _____

Date: _____

A B C D

(4 3 8 2)
 6 5 10 (4)
 12 9 16 18

Now find
 smallest
 element from
 this.
 Here it is
 3.
 Add in final
 list.

Final
 list

2

3

4

4

5

6

8

9

10

12

16

18

A B C D

(4 3 8 2)
 6 5 10 (4)
 12 9 16 18

Now
 here
 smallest is 4; There are
 two 4 so we take first
 one.

A B C D

4 3 8 2
 (6 5 10 4)
 12 9 16 18

Now again
 4 is smallest.

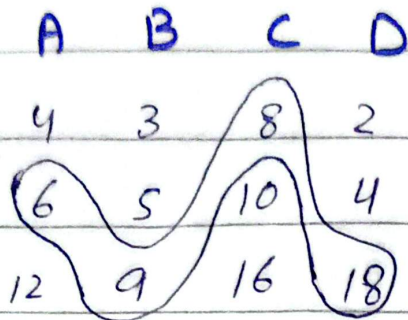
A B C D

4 3 8 2
 (6 5 10 4)
 12 9 16 18

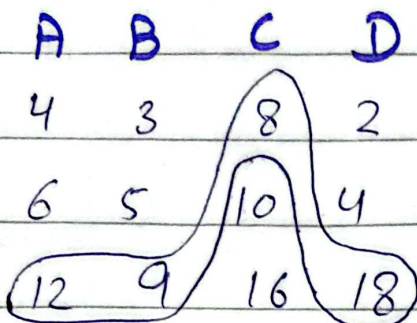
Here 5
 is smallest.

Day: _____

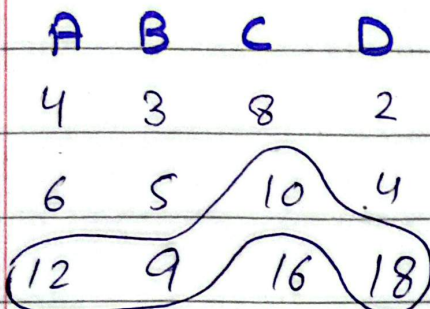
Date: _____



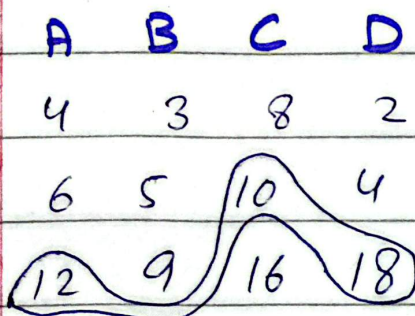
• 6 is smallest.



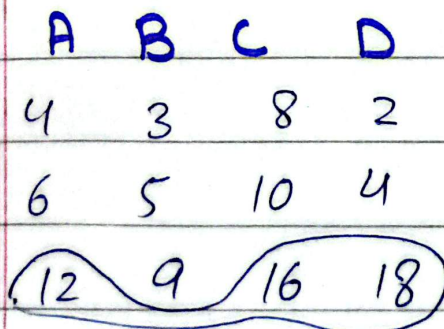
• 8 is smallest.



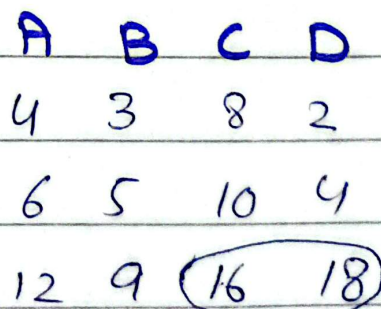
• 9 is smallest.



• 10 is smallest.



↓
12 is smallest.



• 16 is smallest and then 8.

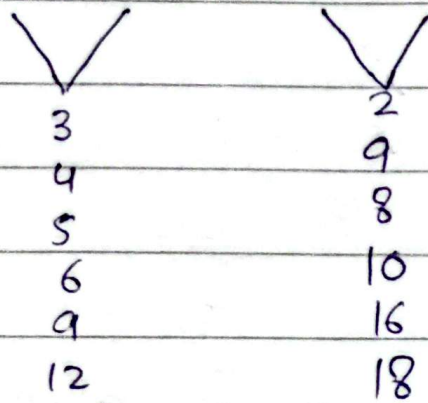
Day: _____

Date: _____

- If we merge only two list at a time, it is called as 2-way Merging. (Mostly we use this method).
- Let see an example that if we have 4-list how we solve them using 2-way Merging.

Method 01:

A	B	C	D
4	3	8	2
6	5	10	4
12	9	16	18

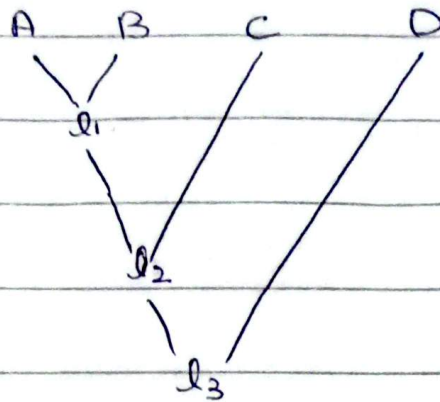


Final sorted list
2, 3, 4, 4, 5, 6, 8, 9, 10, 12, 16, 18

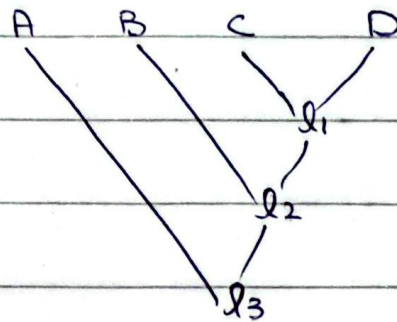
Day: _____

Date: _____

Method 02:



Method 03:

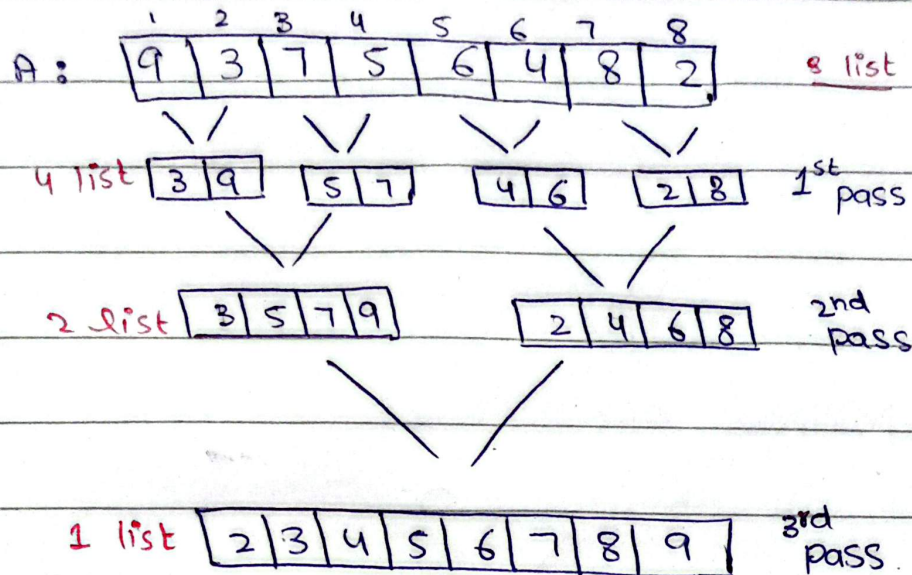


- There are Two merge Sort methods:
 - 01) merge Sort
 - 02) 2-Way Merge Sort

Difference b/w these:

- 2-way Merge Sort is iterative process using loop.
- Merge Sort is a recursive algorithm.

2-way Merge Sort:



If we have only one list and that list is not sorted as in above example, then

Assume that each element in that list is a list meaning above example has 8 lists in start.

Time Complexity:

$$\frac{8}{2^3} = 1$$

$$8 = 2^3$$

$$\log_2 8 = 3$$

Total time complexity: $O(n \log n)$.

Day: _____

Date: _____

Merge Sort Algorithm:

- ↳ recursive procedure
- ↳ Divide and conquer procedure.

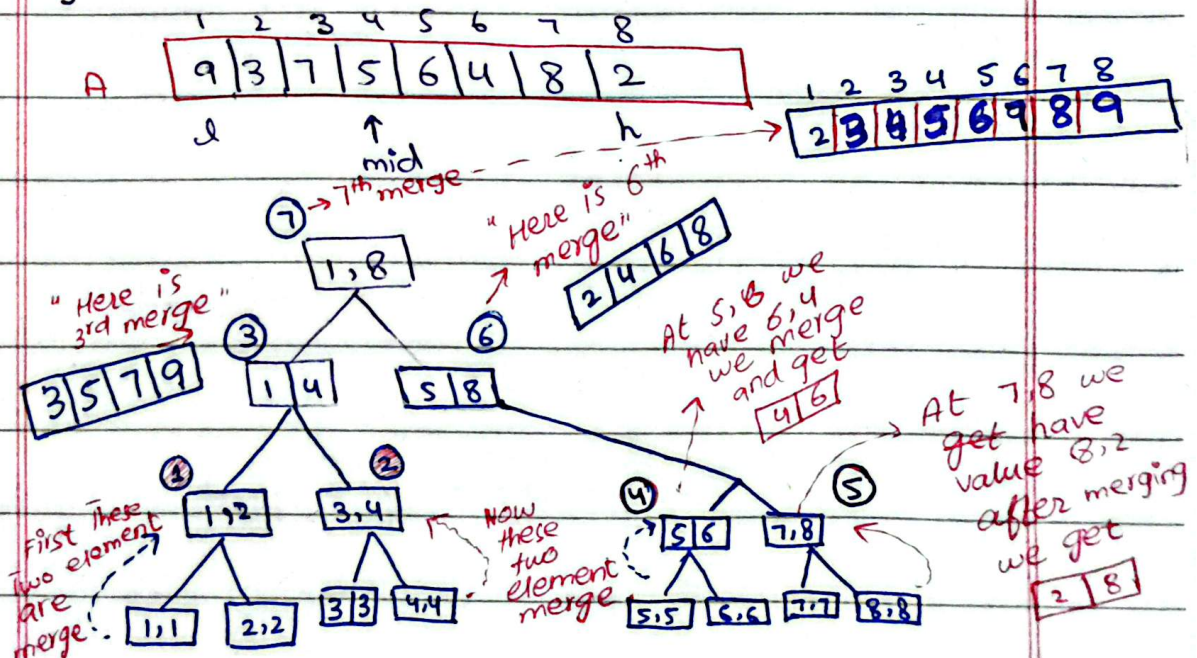
Algo MergeSort (l, h)

```

{
  if (l < h) {
    mid = (l + h) / 2;
    MergeSort(l, mid);
    MergeSort(mid + 1, h);
    Merge(l, mid, h);
  }
}

```

→ "we have seen before how merging is done and also it Algo."



- At 1,2 we have values 9,3 They merge we get

[3, 9]

- At 3,4 we have values 7,5 They merge we get

[5, 7]

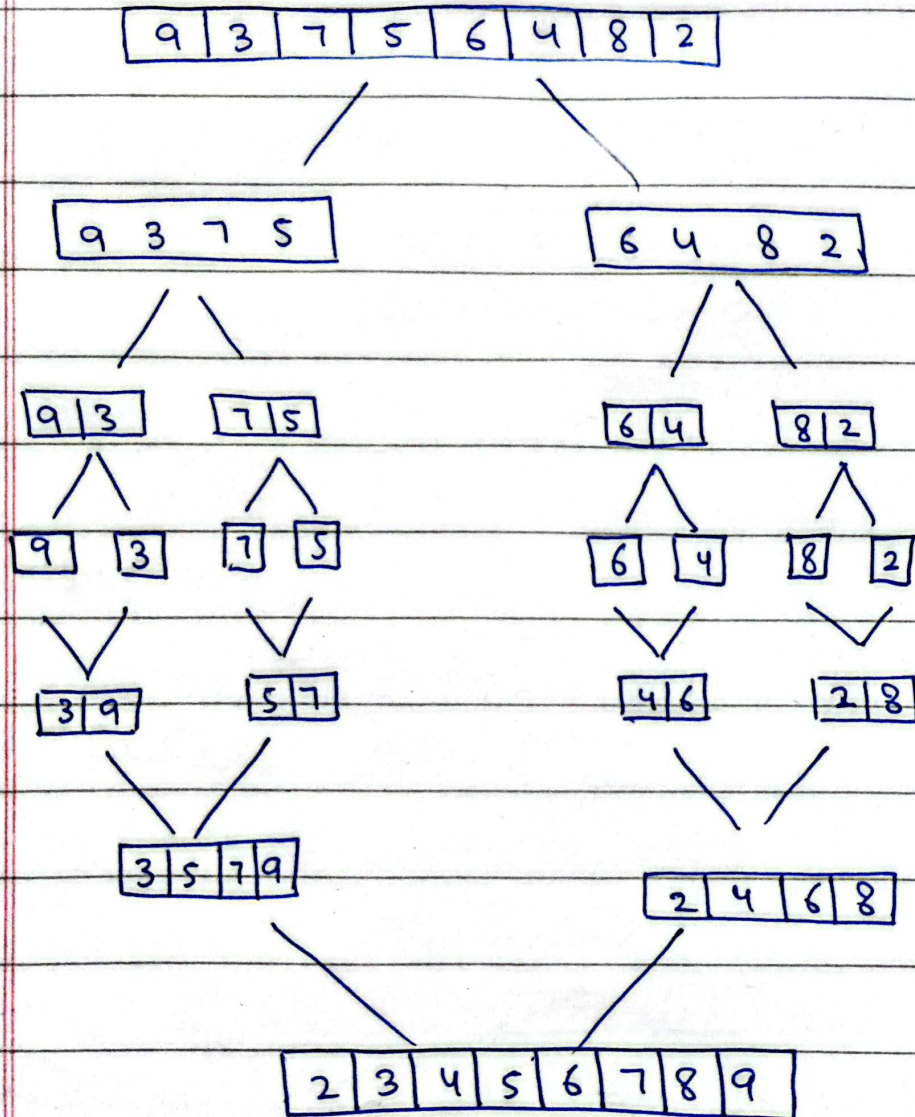
Day: _____

Date: _____

Time Complexity:

$O(n \log n)$

Visualization of Algorithm:



- Post order traversal merging is done.

Day: _____

Date: _____

Time Complexity :

$$T(n) = 2T(n/2) + n$$

$$T(n) = \begin{cases} 1 & n=1 \\ 2T(n/2) + n & n>1 \end{cases}$$

By Master theorem;

$$a = 2$$

$$b = 2$$

$$f(n) = n$$

$$\log_b^a = \log_2^2 = 1$$

$$n^k = n^1$$

$$\log_2^2 = k$$

$$O(n \log n).$$

Pros and Cons of Merge Sort:

• Pros

- 01) Support (size ~~list~~) large size list.
- 02) Suitable for linked list
- 03) Support External Sorting.
- 04) Merge sort is stable.

• Cons

- 01) Extra Space (not inplace sort)
- 02) No Small problem
- 03) Recursive

- Merge Sort take $O(n + \log n)$ space.

Day: _____

Date: _____

Quick Sort:

It follows divide and conquer strategy.

10	80	90	60	30	20
----	----	----	----	----	----

see in the list which element is sorted?

10 is sorted.

Similarly in the following lists:

6 3 3 4 2 1 9 ^{sorted}

4 6 7 10 16 12 13 14
_{sorted}

How it works:

01) Pick one element from the list—called the pivot.

02) Rearrange the list so that:

All elements smaller than the

- pivot go to the left.

- All elements greater than the pivot go to the right.

03) Repeat the same process for the left and right parts.

Day: _____

Date: _____

Example:

0	1	2	3	4	5	6	7	8	9
10	16	8	12	15	6	3	9	5	∞
$\uparrow$ i									$\downarrow$ h j

Pivot = 10

- i will search for the elements greater than pivot and j will search for elements smaller than pivot.
- Increment i until you find an element greater than pivot.
- Decrement j until you find any element smaller than pivot.

10	16	8	12	15	6	3	9	5	∞
	i							j	

Exchange positions.

10	5	8	12	15	6	3	9	16	∞
	i							j	

Repeat the step:

Is $8 > 10$ (F)

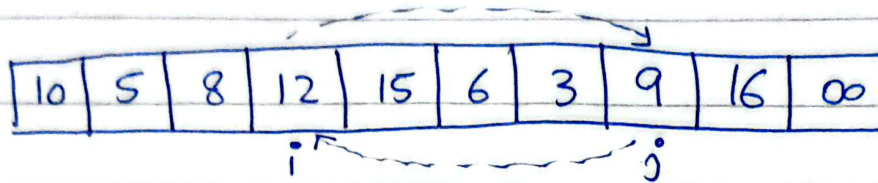
Is $12 > 10$ (T) so $i \rightarrow 12$

Is $9 < 10$ (T) so $j \rightarrow 9$

Interchange positions:

Day: _____

Date: _____

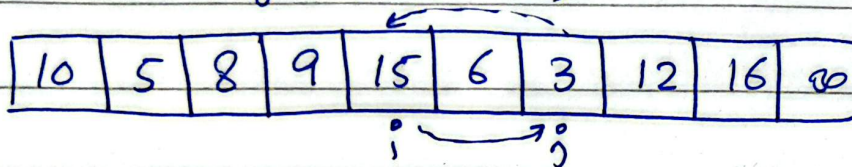


Now again

Is $15 > 10$ (T) $i = 15$

Is $3 < 10$ (T) $j = 3$

Interchange positions;



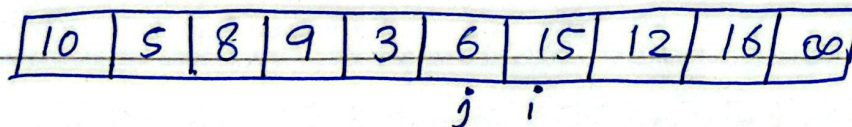
Now again:

Is $6 > 10$ (F)

Is $15 > 10$ (T) $i = 15$

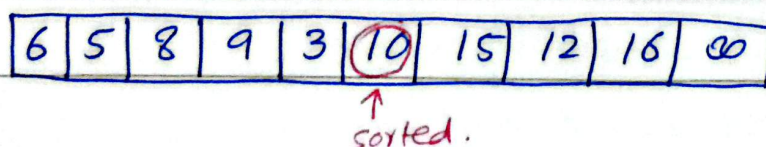
Is $6 < 10$ (T) $j = 6$

Interchange position:



- As $i > j$; so we find position of pivot which is at j , in this case 6.

Replace 6 with pivot.



Day: _____

Date: _____

- Now do the same thing recursively on left and right side elements until the list is sorted.

Algo partition(l, h)

```
{
    Pivot = A[l];
    i = l; j = h;
    while (i < j) {
        do
        {
            i++;
        }
        while (A[i] ≤ pivot);
        do
        {
            j--;
        }
        while (A[j] > pivot);
        if (i < j)
            swap(A[i], A[j]);
    }
    swap(A[l], A[j]);
    return j;
}
```

Algo QuickSort(l, h) {

```
    if (l < h)
    {
        j = Partition(l, h);
        QuickSort(l, j);
        QuickSort(j+1, h);
    }
}
```

}

Day: _____

Date: _____

Time complexity:

- Best case - $O(n \log n)$ → this is done only if the partition occurs through middle everytime.

Achieving best case in quick sort is not possible it may randomly happen.

- Worst case - $O(n^2)$ → this will happen when the list is already sorted and partition is done from start.

Two suggestions for improving the worst case of quick sort:

- 01) Select middle element as pivot.
- 02) select Random element as pivot.

Day: _____

Date: _____

Strassen's Matrix Multiplication:

↳ follows Divide and conquer approach.

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}_{\substack{m \times n \\ 2 \times 2}} \times B = \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix}_{2 \times 2} = C = \begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix}_{2 \times 2}$$

$$C_{ij} = \sum_{k=1}^n A_{ik} * B_{kj}$$

Algo for Multiply 2 Matrix

for (i=0 i<n i++)

for (j=0 j<n j++)

C[i,j] = 0

for (k=0 k<n k++)

C[i,j] = A[i,k] * B[k,j]

- Time complexity $\rightarrow O(n^3)$.

Now let's apply divide and conquer strategy to solve this (Multiply Matrix).

- let see what we can do to solve or multiply matrixes using divide and conquer?
- What divide and conquer say that break the problem into small, then solve and combine to get original solution.
- In this condition what would be the small problem after which we can solve.

if the matrix is 2×2 it is a small problem, we can solve it.

if the matrix is 4×4 or 8×8 we can divide it into small problem and then solve but if it is 2×2 we directly solve it.

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \times B = \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = C = \begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix}$$

How we get the each term of resultant matrix?

$$C_{11} = a_{11} * b_{11} + a_{12} * b_{21}$$

$$C_{12} = a_{11} * b_{12} + a_{12} * b_{22}$$

$$C_{21} = a_{21} * b_{11} + a_{22} * b_{21}$$

$$C_{22} = a_{21} * b_{12} + a_{22} * b_{22}$$

if we have 2×2 matrixes then we can do multiply simply by using these formulas.

Here for multiplying two matrixes we are not using any (3 for loops) here, but directly using formula.

So if we say time taken by this how much time is taken;

Day: _____

Date: _____

consider each statement is taken 1 unit of time. Then Total it is taken 4 unit of time; That is constant time.

Now This is define as The small problem.

Now what if There is a large problem like having 4×4 matrixes.

↳ first we dividet it into smaller and than solve them.

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{bmatrix} \quad B = \begin{bmatrix} b_{11} & b_{12} & b_{13} & b_{14} \\ b_{21} & b_{22} & b_{23} & b_{24} \\ b_{31} & b_{32} & b_{33} & b_{34} \\ b_{41} & b_{42} & b_{43} & b_{44} \end{bmatrix}$$

(4x4) (4x4)

→ if we dide it by 2;

$\frac{4}{2} \times \frac{4}{2}$ (it divide each matrix in 4 2×2 matrixes).

$$A = \begin{array}{|cc|cc|} \hline a_{11} & a_{12} & a_{13} & a_{14} \\ \hline a_{21} & a_{22} & a_{23} & a_{24} \\ \hline a_{31} & a_{32} & a_{33} & a_{34} \\ \hline a_{41} & a_{42} & a_{43} & a_{44} \\ \hline \end{array} \quad B = \begin{array}{|cc|cc|} \hline b_{11} & b_{12} & b_{13} & b_{14} \\ \hline b_{21} & b_{22} & b_{23} & b_{24} \\ \hline b_{31} & b_{32} & b_{33} & b_{34} \\ \hline b_{41} & b_{42} & b_{43} & b_{44} \\ \hline \end{array}$$

$\begin{matrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{matrix}$

 $\begin{matrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{matrix}$

if we consider each as an element like

4 elements in A ($A_{11}, A_{12}, A_{21}, A_{22}$) then
 what? It becomes 2×2 matrix.
 and we can solve this.

How?

$\Rightarrow \left. \begin{array}{l} A_{11} \text{ with } B_{11} \\ A_{12} \text{ with } B_{21} \end{array} \right\} \rightarrow \text{and then add them.}$

we follow the same formula for
 multiple these matrixes.

Algorithm for multiply Two
 matrixes using DAC:

Algo $MM(A, B, n) \{$

if ($n \leq 2$) {

$C = 4$ formulas

"if there is simple
 2×2 matrix, simple
 multiply by formulas
 as we discuss before."

}
 else {

mid $\leftarrow n/2$

$MM(A_{11}, B_{11}, n/2) \oplus MM(A_{12}, B_{21}, n/2)$

$MM(A_{11}, B_{12}, n/2) \oplus MM(A_{12}, B_{22}, n/2)$

$MM(A_{21}, B_{11}, n/2) \oplus MM(A_{22}, B_{21}, n/2)$

$MM(A_{21}, B_{12}, n/2) \oplus MM(A_{22}, B_{22}, n/2)$

}

}

"This is not simple
 integer addition
 it is Matrix
 addition."

Day: _____

Date: _____

Time Complexity :

$$T(n) = \begin{cases} 1 & n \leq 2 \\ 8T(n/2) + n^2 & n > 2 \end{cases}$$

By applying master theorem;

$$\begin{aligned} a &= 8 & \log_b^a &= \log_2^8 = 3 \\ b &= 2 \\ f(n) &= n^2 & n^k &= n^2 \end{aligned}$$

$$\Rightarrow \Theta(n^3)$$

But if say say for space as it is doing work recursively and use stack so it uses extra space so if we see space then multiplication using matrix is more good.

Now let's see what Strassen's has done?

He reduces the number of multiplications, as addition and subtraction take less time than multiplication.

Strassen's formula if we see number

of multiplications are reduced no doubt but addition and subtractions are increase.

Here are his formulas:

$$P = (A_{11} + A_{22})(B_{11} + B_{22})$$

$$Q = (A_{21} + A_{22})B_{11}$$

$$R = A_{11}(B_{12} - B_{22})$$

$$S = A_{22}(B_{21} - B_{11})$$

$$T = (A_{11} + A_{12})B_{22}$$

$$U = (A_{21} - A_{11})(B_{11} + B_{12})$$

$$V = (A_{12} - A_{22})(B_{21} + B_{22})$$

$$C_{11} = P + S - T + V$$

$$C_{12} = R + T$$

$$C_{21} = Q + S$$

$$C_{22} = P + R - Q + U$$

By using these formulas, ~~when~~

The Algo will be calling 7 time instead of 8 times.

Day: _____

Date: _____

What will be the recurrence relation for this one?

$$T(n) = \begin{cases} 1 & n \leq 2 \\ 7T(n/2) + n^2 & n \geq 2 \end{cases}$$

$$\log_2 7 = 2.81 \quad k=2$$

$$O(n^{\log_2 7}) = O(n^{2.81})$$

↑
Strassen has
reduce time
complexity

The time complexity is $n^{2.81}$ which is little less than n^3 .

This is "Strassen's Multiplications."